Employee Payroll Management System

Chinni Bhanu Prakash
Kondapali Uday Kumar

Problem Statement & Objectives

Problem Statement - An organization needs a reliable database to manage employee details, departments, attendance and monthly payroll. Manual records make salary calculations, attendance tracking, deductions and reporting difficult. The system organizes payroll data, maintains history and supports accurate HR and finance reports.

Objectives:

- To design an efficient Employee Payroll database.
- To manage employee and department information.
- To record daily attendance and overtime.
- To calculate monthly salaries using allowances and deductions.
- To maintain payroll history and payment status.
- To generate useful employee and salary reports.

Database Design

DEPARTMENTS (DEPT_ID PK, DEPT_NAME, LOCATION)

EMPLOYEES (EMP_ID PK, EMP_NAME, PHONE, EMAIL, JOB_TITLE, HIRE_DATE, BASIC_SALARY, DEPT_ID FK, EMPLOYMENT_STATUS)

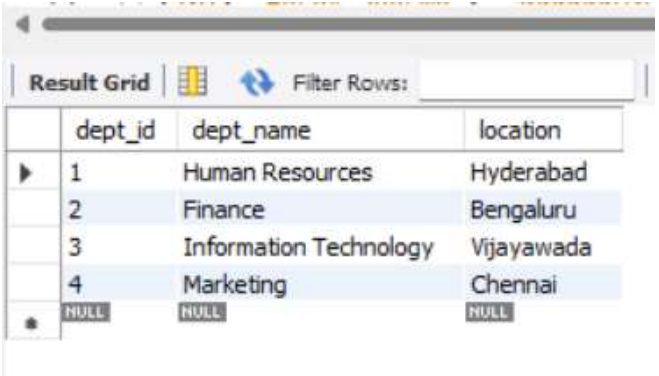
ATTENDANCE (ATTENDANCE_ID PK, EMP_ID FK, ATTENDANCE_DATE, ATTENDANCE_STATUS, OVERTIME_HOURS)

PAYROLL (PAYROLL_ID PK, EMP_ID FK, PAY_MONTH, BASIC_SALARY, ALLOWANCES, DEDUCTIONS, NET_SALARY, PAYMENT_DATE, PAYMENT_STATUS)

Sample Data

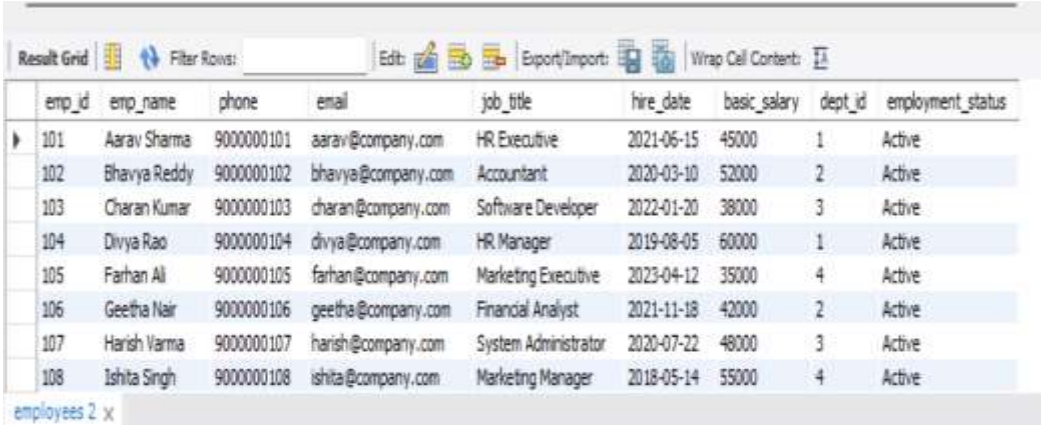
CREATE TABLE Departments
(DEPT_ID INT PRIMARY KEY,
DEPT_NAME VARCHAR(50),
LOCATION VARCHAR(30));

CREATE TABLE Employees
(EMP_ID INT PRIMARY KEY, EMP_NAME VARCHAR(50),
PHONE VARCHAR(15), EMAIL VARCHAR(60), JOB_TITLE VARCHAR(40),
HIRE_DATE DATE, BASIC_SALARY DECIMAL(10,2),
DEPT_ID INT, EMPLOYMENT_STATUS VARCHAR(20),
FOREIGN KEY (DEPT_ID) REFERENCES Departments(DEPT_ID));



Result Grid | Filter Rows:

dept_id	dept_name	location
1	Human Resources	Hyderabad
2	Finance	Bengaluru
3	Information Technology	Vijayawada
4	Marketing	Chennai
NULL	NULL	NULL



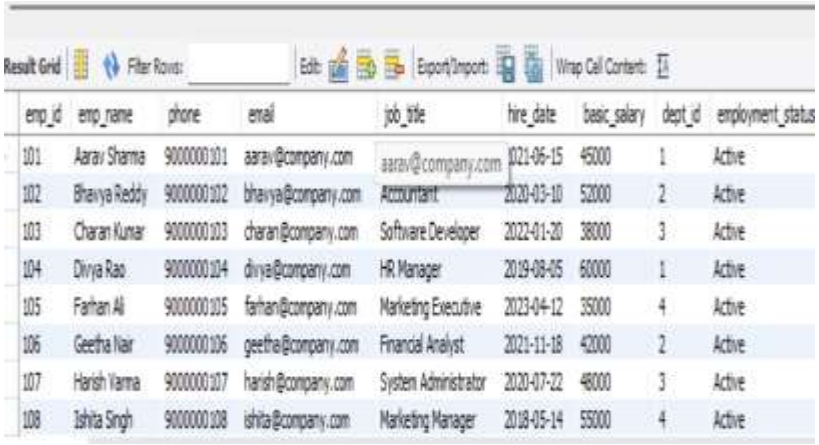
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	phone	email	job_title	hire_date	basic_salary	dept_id	employment_status
101	Aarav Sharma	9000000101	aarav@company.com	HR Executive	2021-06-15	45000	1	Active
102	Bhavya Reddy	9000000102	bhavya@company.com	Accountant	2020-03-10	52000	2	Active
103	Charan Kumar	9000000103	charan@company.com	Software Developer	2022-01-20	38000	3	Active
104	Divya Rao	9000000104	divya@company.com	HR Manager	2019-08-05	60000	1	Active
105	Farhan Ali	9000000105	farhan@company.com	Marketing Executive	2023-04-12	35000	4	Active
106	Geetha Nair	9000000106	geetha@company.com	Financial Analyst	2021-11-18	42000	2	Active
107	Harish Varma	9000000107	harish@company.com	System Administrator	2020-07-22	48000	3	Active
108	Ishita Singh	9000000108	ishita@company.com	Marketing Manager	2018-05-14	55000	4	Active

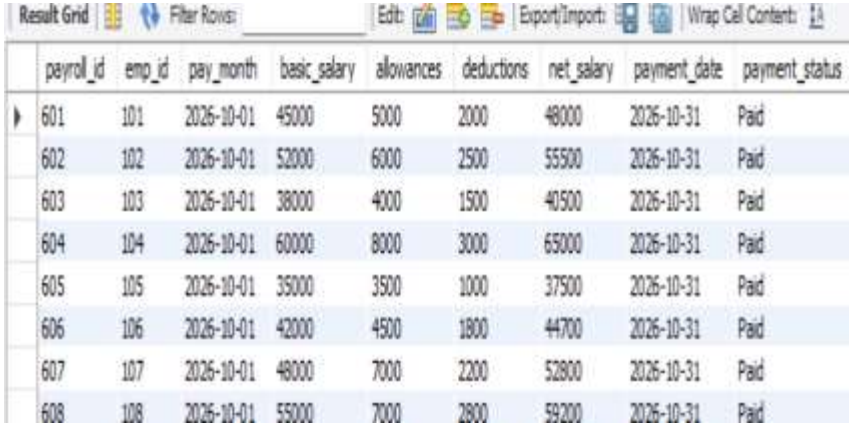
employees 2 x

CREATE TABLE Attendance
(ATTENDANCE_ID INT PRIMARY KEY,
EMP_ID INT, ATTENDANCE_DATE DATE,
ATTENDANCE_STATUS VARCHAR(20),
OVERTIME_HOURS DECIMAL(4,1),
FOREIGN KEY (EMP_ID) REFERENCES Employees(EMP_ID));

CREATE TABLE Payroll
(PAYROLL_ID INT PRIMARY KEY, EMP_ID INT,
PAY_MONTH DATE, BASIC_SALARY DECIMAL(10,2),
ALLOWANCES DECIMAL(10,2), DEDUCTIONS DECIMAL(10,2),
NET_SALARY DECIMAL(10,2), PAYMENT_DATE DATE,
PAYMENT_STATUS VARCHAR(20),
FOREIGN KEY (EMP_ID) REFERENCES Employees(EMP_ID));

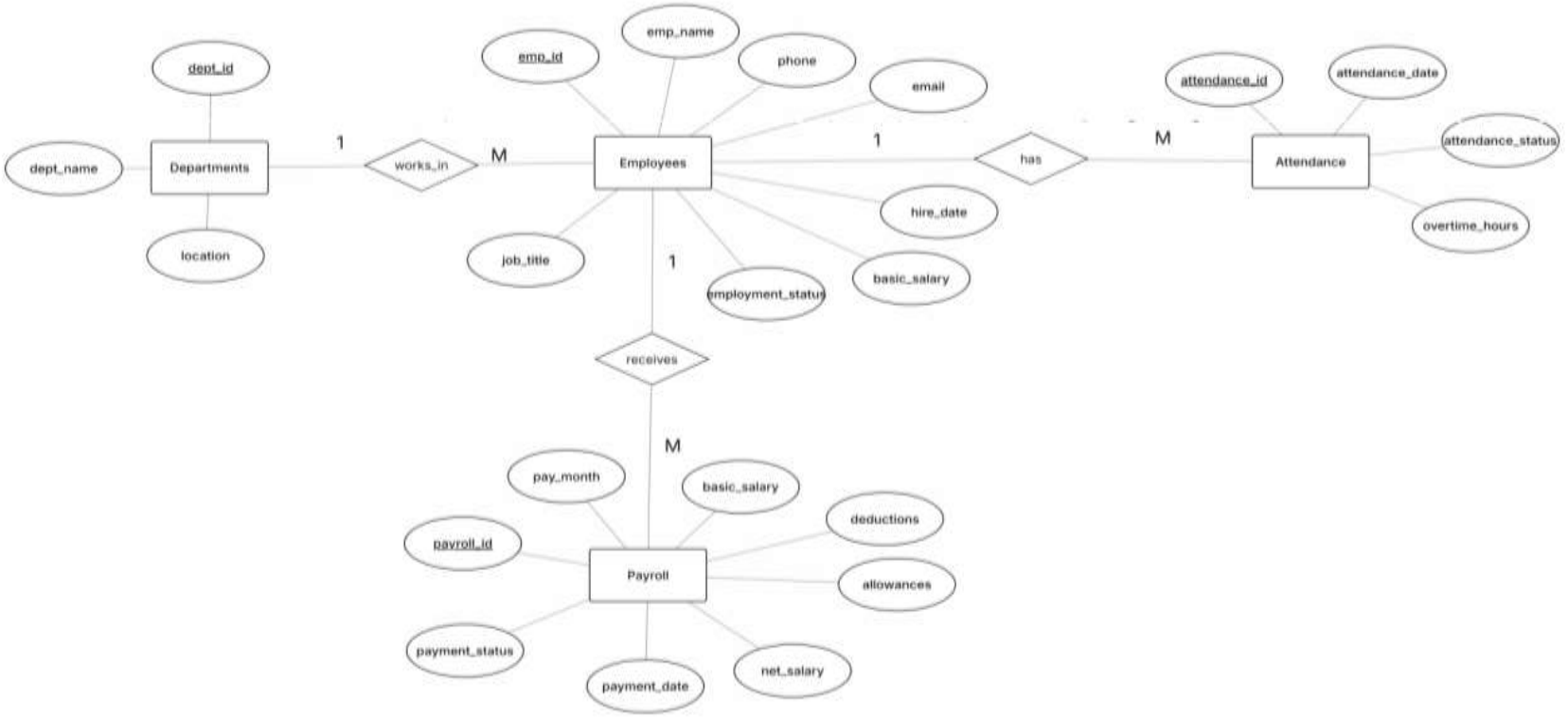


emp_id	emp_name	phone	email	job_title	hire_date	basic_salary	dept_id	employment_status
101	Aarav Sharma	9000000101	aarav@company.com	aarav@company.com	1021-06-15	45000	1	Active
102	Bhavya Reddy	9000000102	bhavya@company.com	Accountant	2020-03-10	52000	2	Active
103	Charan Kumar	9000000103	charan@company.com	Software Developer	2022-01-20	38000	3	Active
104	Divya Rao	9000000104	divya@company.com	HR Manager	2019-08-05	60000	1	Active
105	Farhan Ali	9000000105	farhan@company.com	Marketing Executive	2023-04-12	35000	4	Active
106	Geetha Nair	9000000106	geetha@company.com	Financial Analyst	2021-11-18	42000	2	Active
107	Harish Varma	9000000107	harish@company.com	System Administrator	2020-07-22	48000	3	Active
108	Ishita Singh	9000000108	ishita@company.com	Marketing Manager	2018-05-14	55000	4	Active



payroll_id	emp_id	pay_month	basic_salary	allowances	deductions	net_salary	payment_date	payment_status
601	101	2026-10-01	45000	5000	2000	48000	2026-10-31	Paid
602	102	2026-10-01	52000	6000	2500	55500	2026-10-31	Paid
603	103	2026-10-01	38000	4000	1500	40500	2026-10-31	Paid
604	104	2026-10-01	60000	8000	3000	65000	2026-10-31	Paid
605	105	2026-10-01	35000	3500	1000	37500	2026-10-31	Paid
606	106	2026-10-01	42000	4500	1800	44700	2026-10-31	Paid
607	107	2026-10-01	48000	7000	2200	52800	2026-10-31	Paid
608	108	2026-10-01	55000	7000	2800	59200	2026-10-31	Paid

ER Diagram


dept_id
dept_name
Departments
location
1
works_in
M
emp_id
emp_name
phone
email
Employees
job_title
employment_status
basic_salary
hire_date
1
has
M
attendance_id
attendance_date
attendance_status
overtime_hours
Attendance
1
receives
M
pay_month
basic_salary
deductions
payroll_id
Payroll
allowances
payment_status
payment_date
net_salary

CRUD Operations queries

C – CREATE (INSERT) – Add payroll employee records

INSERT INTO Employees VALUES

(101,'Aarav Sharma','9000000101','aarav@company.com','HR Executive','2021-06-15',45000,1,'Active'),

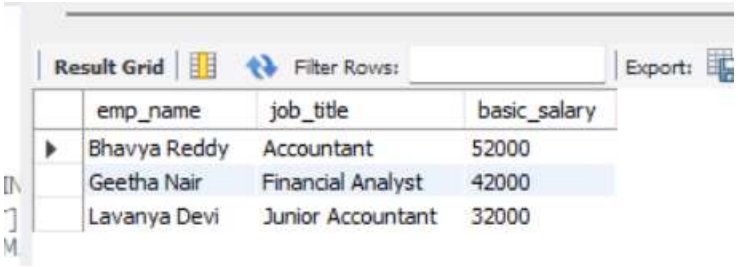
(102,'Bhavya Reddy','9000000102','bhavya@company.com','Accountant','2020-03-10',52000,2,'Active');

R – READ (SELECT) – Retrieve employees

SELECT emp_name, job_title, basic_salary

FROM Employees

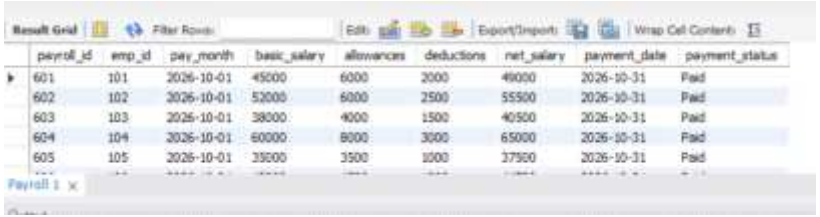
WHERE dept_id = 2;



CRUD Operations queries

U – UPDATE – Modify payroll details

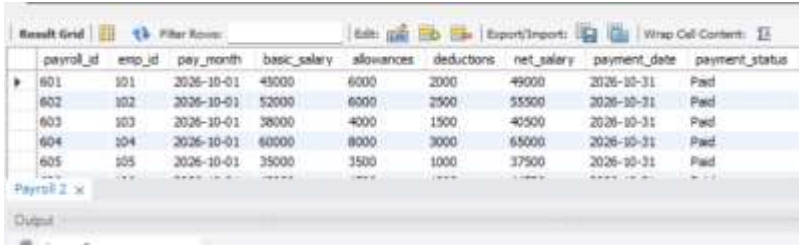
```
UPDATE Payroll
SET allowances = allowances + 1000,
    net_salary = net_salary + 1000
WHERE payroll_id = 601;
```



payroll_id	emp_id	pay_month	basic_salary	allowances	deductions	net_salary	payment_date	payment_status
601	101	2026-10-01	45000	6000	2000	49000	2026-10-31	Paid
602	102	2026-10-01	52000	6000	2500	55500	2026-10-31	Paid
603	103	2026-10-01	38000	4000	1500	40500	2026-10-31	Paid
604	104	2026-10-01	60000	8000	3000	65000	2026-10-31	Paid
605	105	2026-10-01	35000	3500	1000	37500	2026-10-31	Paid

D – DELETE – Remove a pending payroll record

```
DELETE FROM Payroll
WHERE payroll_id = 609 AND payment_status = 'Pending';
```



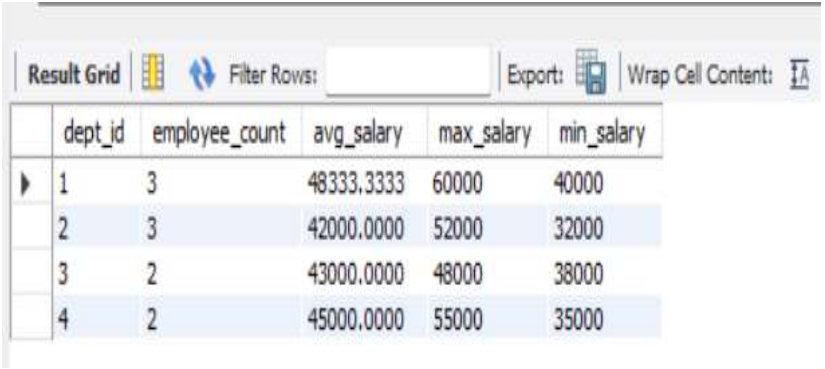
payroll_id	emp_id	pay_month	basic_salary	allowances	deductions	net_salary	payment_date	payment_status
601	101	2026-10-01	45000	6000	2000	49000	2026-10-31	Paid
602	102	2026-10-01	52000	6000	2500	55500	2026-10-31	Paid
603	103	2026-10-01	38000	4000	1500	40500	2026-10-31	Paid
604	104	2026-10-01	60000	8000	3000	65000	2026-10-31	Paid
605	105	2026-10-01	35000	3500	1000	37500	2026-10-31	Paid

Aggregate functions queries

Aggregate functions perform calculations on multiple rows and return a single value per group.

- **COUNT()** – Counts records.
- **SUM()** – Adds values.
- **AVG()** – Calculates average.
- **MAX()** – Returns highest value.
- **MIN()** – Returns lowest value.

```
SELECT dept_id, COUNT(*) AS employee_count,
    AVG(basic_salary) AS avg_salary,
    MAX(basic_salary) AS max_salary,
    MIN(basic_salary) AS min_salary
FROM Employees
GROUP BY dept_id;
```



dept_id	employee_count	avg_salary	max_salary	min_salary
1	3	48333.3333	60000	40000
2	3	42000.0000	52000	32000
3	2	43000.0000	48000	38000
4	2	45000.0000	55000	35000

String functions queries

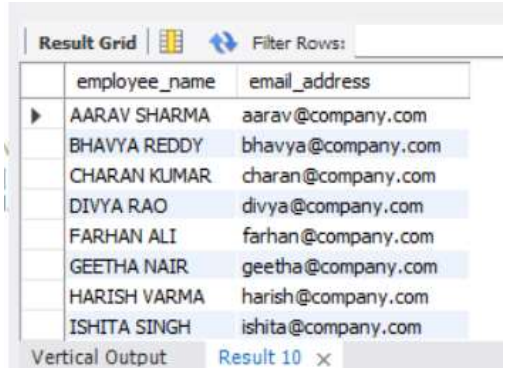
String functions transform text values in database columns.

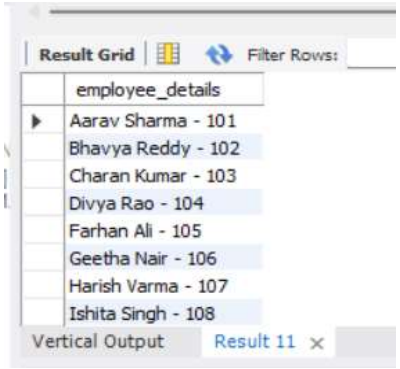
- **UPPER()** – Converts text to uppercase.
- **LOWER()** – Converts text to lowercase.
- **CONCAT()** – Combines strings.
- **LENGTH()** – Returns string length.
- **LEFT()** – Extracts characters from the beginning.

```
SELECT UPPER(emp_name) AS employee_name,
    LOWER(email) AS email_address
FROM Employees;

SELECT CONCAT(emp_name, ' - ', emp_id) AS employee_details
FROM Employees;

SELECT LENGTH(emp_name) AS name_length,
    LEFT(emp_name, 4) AS name_prefix
FROM Employees;
```





Date functions queries

Date functions retrieve and calculate date/time values.

- **CURDATE()** – Current date.
- **NOW()** – Current date and time.
- **DAYNAME()** – Weekday name.
- **MONTHNAME()** – Month name.
- **TIMESTAMPDIFF()** – Difference between dates.

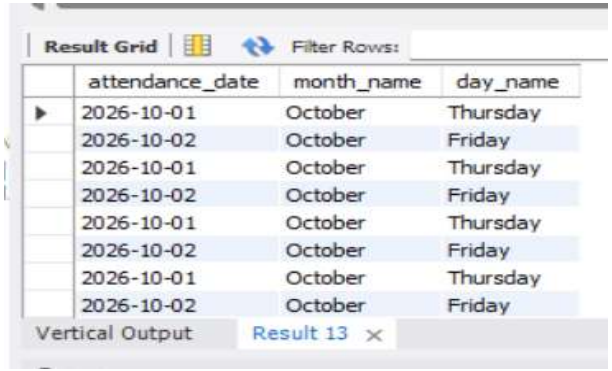
```
SELECT emp_name,
    TIMESTAMPDIFF(YEAR, hire_date, CURDATE()) AS experience_years
FROM Employees;
```

```
SELECT attendance_date,
    MONTHNAME(attendance_date) AS month_name,
    DAYNAME(attendance_date) AS day_name
FROM Attendance;
```


Result Grid | Filter Rows:

emp_name	experience_years
Aarav Sharma	5
Bhavya Reddy	6
Charan Kumar	4
Divya Rao	7
Farhan Ali	3
Geetha Nair	4
Harish Varma	6
Ishita Singh	8

Vertical Output | Result 12


Result Grid | Filter Rows:

attendance_date	month_name	day_name
2026-10-01	October	Thursday
2026-10-02	October	Friday
2026-10-01	October	Thursday
2026-10-02	October	Friday
2026-10-01	October	Thursday
2026-10-02	October	Friday
2026-10-01	October	Thursday
2026-10-02	October	Friday

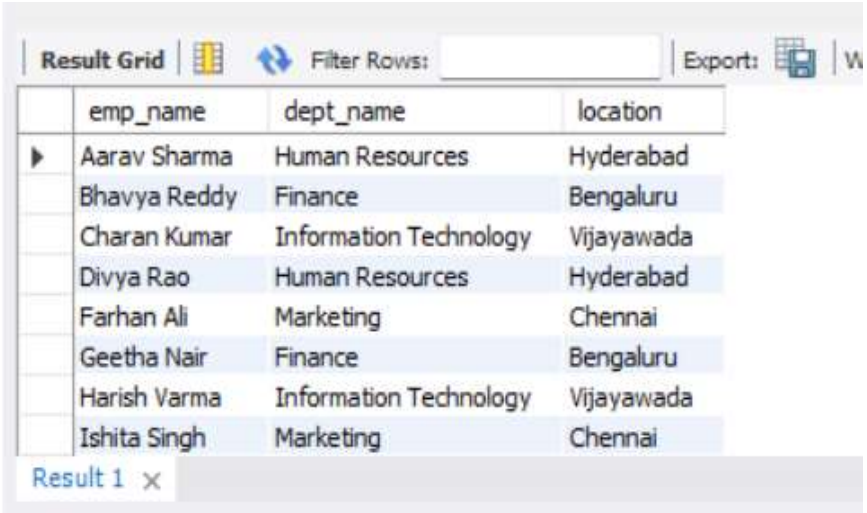
Vertical Output | Result 13

Joins

Used to combine data from two or more tables based on a related column between them.

- INNER JOIN – Only matching records
- LEFT JOIN – All left + matching right
- RIGHT JOIN – All right + matching left
- FULL JOIN – All matching and non-matching records
- CROSS JOIN – All possible combinations
- SELF JOIN – Joins a table with itself

```
SELECT e.emp_name, d.dept_name, d.location
FROM Employees e
INNER JOIN Departments d
ON e.dept_id = d.dept_id;
```



emp_name	dept_name	location
Aarav Sharma	Human Resources	Hyderabad
Bhavya Reddy	Finance	Bengaluru
Charan Kumar	Information Technology	Vijayawada
Divya Rao	Human Resources	Hyderabad
Farhan Ali	Marketing	Chennai
Geetha Nair	Finance	Bengaluru
Harish Varma	Information Technology	Vijayawada
Ishita Singh	Marketing	Chennai

Multiple Joins

Multiple joins allow you to combine data from more than one table in a single query.

```
select e.emp_name, d.dept_name, p.pay_month, p.basic_salary, p.allowances, p.deductions, p.net_salary from employees e
        join Departments d ON e.dept_id = d.dept_id JOIN Payroll p ON e.emp_id = p.emp_id;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	dept_name	pay_month	basic_salary	allowances	deductions	net_salary
Aarav Sharma	Human Resources	2026-10-01	45000	5000	2000	48000
Bhavya Reddy	Finance	2026-10-01	52000	6000	2500	55500
Charan Kumar	Information Technology	2026-10-01	38000	4000	1500	40500
Divya Rao	Human Resources	2026-10-01	60000	8000	3000	65000
Farhan Ali	Marketing	2026-10-01	35000	3500	1000	37500
Geetha Nair	Finance	2026-10-01	42000	4500	1800	44700
Harish Varma	Information Technology	2026-10-01	48000	7000	2200	52800
Ishita Singh	Marketing	2026-10-01	55000	7000	2800	59200

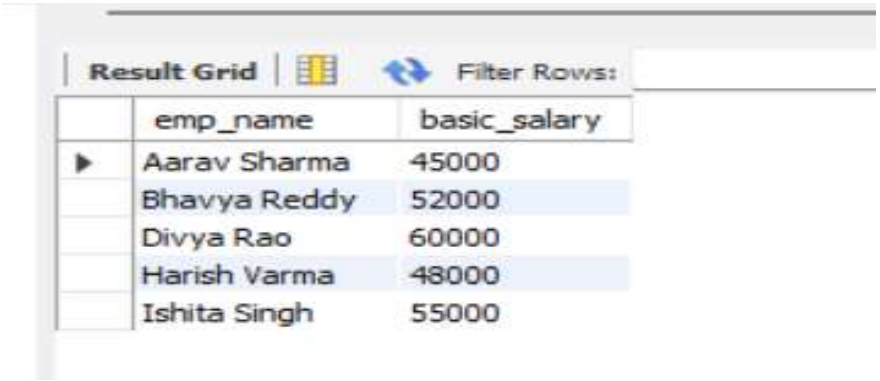
Result 2

Sub Queries

A Subquery is a query written inside another SQL query. The result of the inner query is used by the outer query to retrieve or manipulate data.

- Single-Row Subquery – Returns only one row from the inner query
- Multiple-Row Subquery – Returns more than one row from the inner query.
- Correlated Subquery – Depends on the outer query and executes logically for each row processed by it.

```
SELECT emp_name, basic_salary
FROM Employees
WHERE basic_salary >
    (SELECT AVG(basic_salary)
     FROM Employees);
```

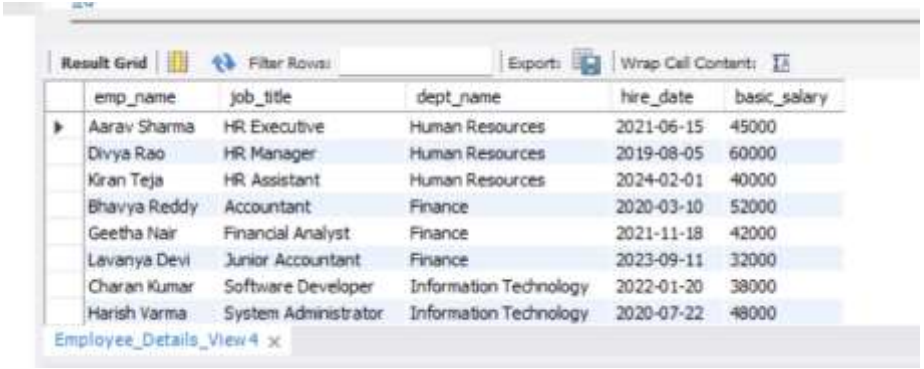


Views

Views in SQL are virtual tables created using SQL queries. They display data from one or more existing tables without storing the actual data separately. Views help simplify complex queries and provide easy access to specific data.

```
CREATE VIEW Employee_Details_View AS
SELECT e.emp_name, e.job_title,
    d.dept_name, e.hire_date, e.basic_salary
FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id;

SELECT * FROM Employee_Details_View;
```



emp_name	job_title	dept_name	hire_date	basic_salary
Aarav Sharma	HR Executive	Human Resources	2021-06-15	45000
Divya Rao	HR Manager	Human Resources	2019-08-05	60000
Kiran Teja	HR Assistant	Human Resources	2024-02-01	40000
Bhavya Reddy	Accountant	Finance	2020-03-10	52000
Geetha Nair	Financial Analyst	Finance	2021-11-18	42000
Lavanya Devi	Junior Accountant	Finance	2023-09-11	32000
Charan Kumar	Software Developer	Information Technology	2022-01-20	38000
Harish Varma	System Administrator	Information Technology	2020-07-22	48000

Transaction Control Language

Used to manage database transactions.

- COMMIT – Permanently saves all changes made in a transaction.
- ROLLBACK – Undoes changes made in the current transaction that have not been committed.
- SAVEPOINT – Creates a temporary point within a transaction to which we can roll back.

```
start transaction;

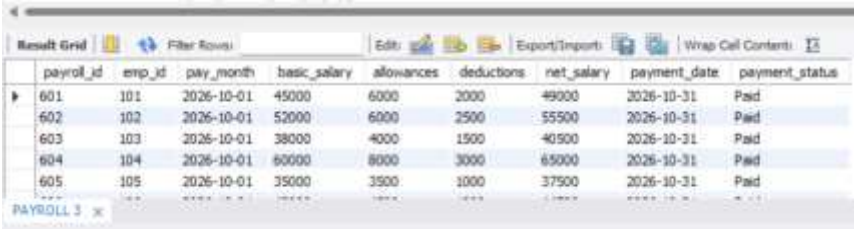
update payroll set allowances = allowances + 2000 ,
net_salary = net_salary + 2000 where emp_id = 107;

savepoint sp1;

update payroll set allowances = allowances + 2000 ,
net_salary = net_salary + 2000 where emp_id = 109;

rollback to sp1;

commit;
```



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

payroll_id	emp_id	pay_month	basic_salary	allowances	deductions	net_salary	payment_date	payment_status
601	101	2026-10-01	45000	6000	2000	49000	2026-10-31	Paid
602	102	2026-10-01	52000	6000	2500	55500	2026-10-31	Paid
603	103	2026-10-01	38000	4000	1500	40500	2026-10-31	Paid
604	104	2026-10-01	60000	8000	3000	65000	2026-10-31	Paid
605	105	2026-10-01	35000	3500	1000	37500	2026-10-31	Paid

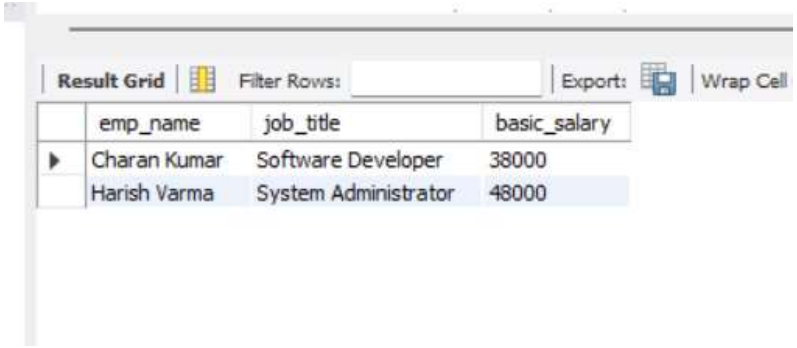
PAYROLL 3

Stored Procedures

A stored procedure is reusable SQL code saved in the database. It can accept parameters and execute a sequence of SQL statements when called.

```
DELIMITER //
CREATE PROCEDURE Get_Employees_By_Dept(IN p_dept_id INT)
BEGIN
  SELECT emp_name, job_title, basic_salary
  FROM Employees
  WHERE dept_id = p_dept_id;
END //
DELIMITER ;

CALL Get_Employees_By_Dept(2);
```



Triggers

A trigger is a database object that automatically runs when an INSERT, UPDATE or DELETE operation occurs. Triggers can automate defaults and help maintain data consistency.

```
(emp_id, pay_month, basic_salary, allowances, deductions,
net_salary)
VALUES
(101, '2026-11-01', 30000, NULL, 2000, 30000);

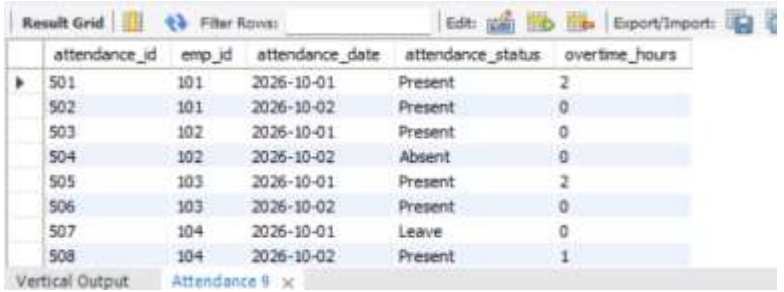
SELECT * FROM Payroll;
```



payroll_id	emp_id	pay_month	basic_salary	allowances	deductions	net_salary	payment_date	payment_status
601	101	2026-10-01	45000	5000	2000	48000	2026-10-31	Paid
602	102	2026-10-01	52000	6000	2500	55500	2026-10-31	Paid
603	103	2026-10-01	38000	4000	1500	40500	2026-10-31	Paid
604	104	2026-10-01	60000	8000	3000	65000	2026-10-31	Paid
605	105	2026-10-01	35000	3500	1000	37500	2026-10-31	Paid
606	106	2026-10-01	42000	4500	1800	44700	2026-10-31	Paid
607	107	2026-10-01	48000	9000	2200	54800	2026-10-31	Paid
608	108	2026-10-01	55000	7000	2800	59200	2026-10-31	Paid

```
INSERT INTO Attendance
(emp_id, attendance_date, attendance_status, overtime_hours)
VALUES
(101, '2026-11-09', NULL, 0);

SELECT * FROM Attendance;
```



attendance_id	emp_id	attendance_date	attendance_status	overtime_hours
501	101	2026-10-01	Present	2
502	101	2026-10-02	Present	0
503	102	2026-10-01	Present	0
504	102	2026-10-02	Absent	0
505	103	2026-10-01	Present	2
506	103	2026-10-02	Present	0
507	104	2026-10-01	Leave	0
508	104	2026-10-02	Present	1

THANK YOU